ECE-374-B: Algorithms and Models of Computation, Summer 2026
Midterm 1

- **You can do hard things!** Grades do matter, but not as much as you may think, but then life is uncertain anyway, so what.
- **Don't cheat.** *You could be needlessly losing the love of your life.* The consequence for cheating is far greater than the reward. Just try your best and you'll be fine.
- **Please read the entire exam before writing anything.** Make sure you check the front and back of all the pages! *Hold the paper like a sandwich — look at the thin edge too. Sometimes there's stuff hiding there!*
- This is a closed-book exam. *Most of this generation doesn't read books anyways.*
- **Do not tear out the cheatsheet!** *Don't cheat, we have to do a bunch of paperwork if you do. 10 years from now, no one will care what you got on this test.*
- You should write your answers legibly and in the space given for the question. *Remember, this is not a job application to become a doctor.*
- **You have** 75 **minutes (**1.25 **hours) for the exam.** Manage your time well. *Don't spend too much time on questions you do not understand and focus on answering as much as you can!*
- Proofs are required only if we specifically ask for them. **You do not need to use proof by induction on any of these questions.**
- Minor notes:
  - You cannot use negation (\) in your regular expressions.
  - Cannot assume the reverse operation is closed for regular or context-free languages (w^R)

Name: _____

NetID: _____

1 Short Answer: Regular Expressions - 16 points

Match each named language L_a–L_e below to one or more of the regular expressions (1–10). Each regular expression may match multiple languages, or none. Use the alphabet $\Sigma = \{a, b\}$. **No reasoning needed. If you erase stuff in the table, please be clear. Don't check an item and then scratch it out in pen.**

Hint: There are eight correct tick marks and some languages have more than one expression while others have only one. Mark the ones that appear the most correct and don't let this suck up too much of your time. **Regular Expressions:**

(1) $(\epsilon + a)(ba)^*(\epsilon + b)$

(2) $\Sigma(\Sigma\Sigma\Sigma)^*$

(3) $aa^*bb^* + bb^*aa^*$

(4) $\Sigma^*b\Sigma\Sigma$

(5) $(b + ba)^*$

(6) $(ab)^*(\epsilon + a) + (ba)^*(\epsilon + b)$

(7) $(aaa + aab + aba + abb + baa + bab + bba + bbb)^*(a + b)$

(8) $(bb^*(\epsilon + a))^*$

(9) $\Sigma^*bb\Sigma^*$

(10) $\Sigma^*ab\Sigma^*$

Language Descriptions:

L_a: All strings w, including ϵ, in which adjacent symbols alternate.

L_b: All strings w whose length modulo 3 is equal to 1.

L_c: All strings w, including ϵ, such that every occurrence of a is immediately preceded by b. Thus strings beginning with a are not included.

L_d: All strings w of length at least 3 whose third symbol from the right is b.

L_e: All strings w that consist of one or more copies of one symbol followed by one or more copies of the other symbol.

Solution:

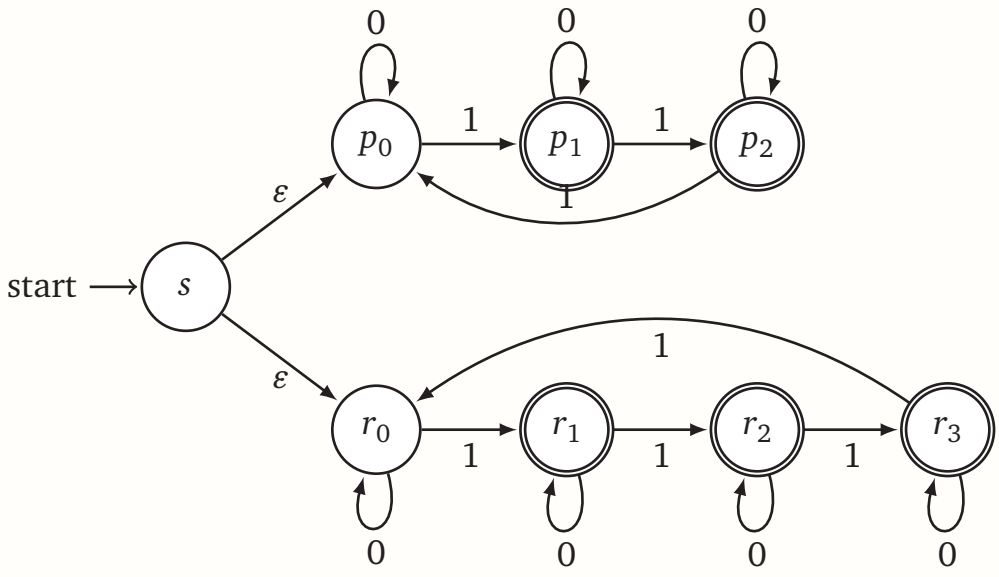
Language	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)
L_a	✓					✓				
L_b		✓					✓			
L_c					✓			✓		
L_d				✓						
L_e			✓							

■

2 Short Answer II - 24 points

a) Give the NFA/DFA that describes the language:
$L = \{w | w \in \Sigma^*$ and the number of 1's in w is NOT divisible by at least one of: 3, 4 $\}$

Solution: We build an NFA that branches into two machines. The top branch accepts if the number of 1s is not divisible by 3. The bottom branch accepts if the number of 1s is not divisible by 4.



■

b) Write the regular expression for L_z defined as:

$$L_x = \{w \in \Sigma^* \mid \text{every } 0 \text{ is immediately followed by at least one } 1\}$$
$$L_y = \{0^n 1^m \mid 0 \le n \le m\}$$

Let $L_z = L_x \cap L_y$.

Solution: Only strings where every 0 is directly followed by a 1 can be in L_x. Only strings of the form $0^n 1^m$ can be in L_y. Only ε and strings with a single 0 followed by a run of 1's satisfy both.
Regular Expression for L_z: $\varepsilon + 01^+$ ■

c) (continuing from part b.) Now suppose we redefine $L_y = \{0^n 1^m \mid 0 \le m \le n\}$. Describe the new language L_z.

Solution: Now, the number of 1's must be less than (or equal to) the number of 0's. There are only two strings from the original regex that follow this pattern, so

$L_z = \varepsilon + 01$. ■

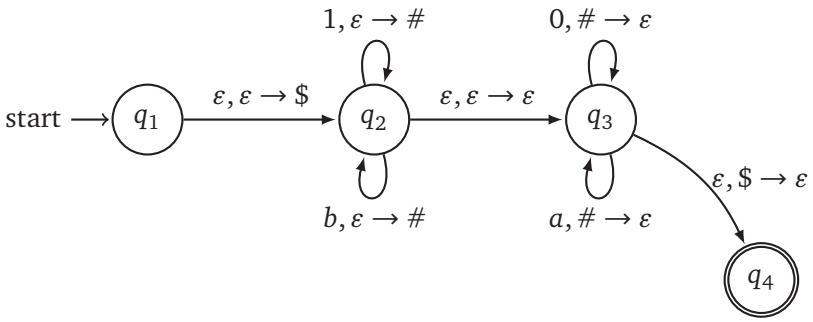
d) Let $L = \{w \mid w \in \Sigma^*, |w| \geq 3, 2 \text{ and } w \text{ has alternating } 0\text{'s and } 1\text{'s}\}$. Prove that any DFA accepting L must have at least two states. Constructing a DFA that accepts L is insufficient. *Hint: Think fooling sets.*

Solution: Let

$$F = \{10, 01\}.$$

The strings are distinguishable since assuming $z = 1$, $10z \in L$ and $01z \notin L$. Therefore, the DFA for L must have atleast two states. ■

e) Consider the language recognized by the PDA below, where $L \subseteq \{0, 1, a, b\}^*$. Give a description (could be a concise english sentence, RegEx or grammar) for L.



Solution: All strings where the number of 1's and b's matches the number of 0's + a's. ■

f) Provide the context-free grammar for the following language:

$$L = \left\{w \mid w \in \{a, b, c, d\}^*,\ w = a^i b^j c^k d^l \text{ where } l < i + j + k\right\}.$$

Solution: A useful grammar is:

$$\begin{aligned}
S &\to aA \mid bB \mid cC \\
A &\to aAD \mid B \mid C \mid D \\
B &\to bBD \mid C \mid D \\
C &\to cCD \mid D \\
D &\to d \mid \varepsilon
\end{aligned}$$

Need to ensure that atleast one a/b/c is created (first rule). Then all future rules allow the creation of at most one d only if a a/b/c is placed. ■

3 Language Transformation - 15 pts

Assume L is a regular language and $\Sigma = \{0, 1\}$. Assume zero-indexing (first bit is at position "[0]"). Prove that the language *ForwardOrBackward(FOB)*$(L) := \{w \mid w \text{ or } w^R \in L\}$ (for every string in L, the $FOB(L)$ must accept that string or it's reverse) is regular.

Solution: We can prove *FOB*(L) is regular by language transformation. Since L is regular, it has a DFA (M). First let's construct a NFA (N^R) to represent the reverse language:

$$
\begin{aligned}
Q^R &= Q \cup \{x\} \\
s^R &= x \\
A^R &= s \\
\delta^R(q_x^R, a) &= q_y^R \text{ for every } \delta(q_x, a) = q_y \\
\delta^R(s^R, 1) &= q_x^R \text{ for every } q_x \in A
\end{aligned}
$$

Now we can construct the NFA (N') for $FOB(L)$ using the DFA M and NFA N_R

$$
\begin{aligned}
Q &= Q \cup Q^R \cup \{q_0'\} \\
s &= q_0' \\
A &= A \cup A^R \\
\delta(q_0', \varepsilon) &= s \\
\delta(q_0', \varepsilon) &= s^R \\
\delta(q, a) &= \delta(q, a) \\
\delta(q, a) &= \delta^R(q, a)
\end{aligned}
$$

Thus, because there is a NFA that can describe *FOB*(L), that language is regular.

■

4 Language classification I (2 parts) - 15 points

Let $\Sigma_4 = \{0, 1\}$ and

$$L_4 = \left\{ww^R | w \in \Sigma_4^*\right\}$$ [1]

a) Is L_4 regular? Indicate whether or not by circling one of the choices below. Either way, prove it.

Solution: regular (not regular)

Suppose we have the fooling set $F = \{0^n 1 | n > 0\}$. Let's take two strings from this set $x = 0^i 1, y = 0^j 1$ and a suffix $z = 10^j$ for all $j \neq i$. Then we know $xz \notin L$ and $yz \in L$ making these two strings distinguishable. Because the fooling set is infinite, it means the DFA that represent L must also be infinite. Contradiction. L must not be regular.

■

b) Is L_4 context-free? Indicate whether or not by circling one of the choices below. Either way, prove it!

Solution: (context-free) not context-free

Yes, it is contextfree, the grammar of L is:

$$S \rightarrow 1S0|1S1|\varepsilon$$

■

[1] Each string is a palindrome with even number of tokens

5 Language classification II (2 parts) - 15 points

Let $\Sigma_5 = \{0, 1\}$ and

$$L_5 = \{xy \in \{0,1\}^n \mid x, y \in \Sigma^* \text{ and } \#_1(x) \text{ and } \#_1(y) \text{ differs by at most } 1\}$$ [2]

a) Is L_5 regular? Indicate whether or not by circling one of the choices below. Either way, prove it.

Solution: (regular) not regular

This one requires you really understanding what the language is. There is no separator in xy and you can choose whatever x and y you want. Therefore, for any string in Σ^*, you can choose a x/y that satisfies this criteria. Hence, this language includes all strings and the language that includes all strings is regular. ■

b) Is L_5 context-free? Indicate whether or not by circling one of the choices below.

Solution: (context-free) not context-free

Every regular language is context free. ■

[2] The $\#_a(w)$ operator counters the number of times character a appears in string w

6 Language classification III (2 parts) - 15 points

Let $\Sigma_{??} = \{0, 1\}$, and define

$$L_{??} = \left\{ xy \mid x, y \in \Sigma^* \text{ where } \begin{array}{l} x \text{ contains at least one } 1\ , \\ y \text{ fulfills: } |y| \geq |x| \end{array} \right\}$$

a) Is L_6 regular? Indicate whether or not by circling one of the choices below. Either way, prove it.

Solution: regular (not regular)

Assume a fooling set $F = \{1^n | n > 0\}$. Take any two arbitrary strings from F ($x = 1^i$, $y = 1^j$) where $j > i$. Assume $z = 0^i$: $xz \in L$ but $yz \notin L$. Therefor F is a fooling set for L_6.

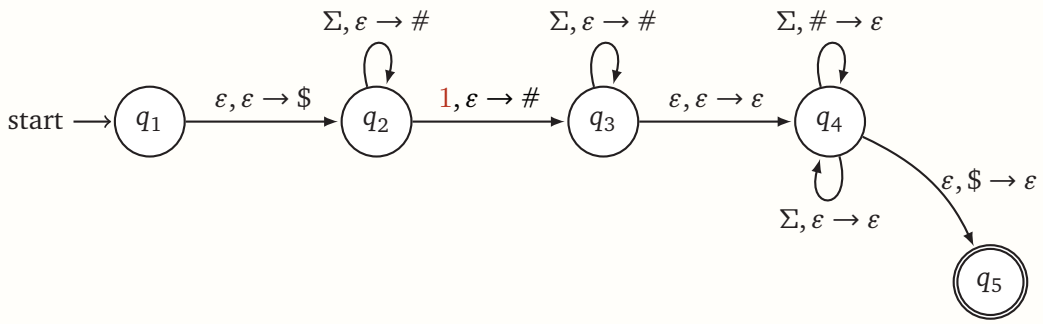
$|F| = \infty$, therefore if L_6 was regular, it would need a DFA with an infinite number of states which is a contradiction.

■

b) Is L_6 context-free? Indicate whether or not by circling one of the choices below. Either way, prove it!

Solution: (context-free) not context-free

A PDA might be the easiest way to show this one:



■

Solution: (context-free) not context-free

An alternative solution is to simply provide a CFG that describes the language. Something liek this should work:

$$S \to 0SY|1AY$$
$$A \to 0AY|1AY|\varepsilon$$
$$Y \to 0|1|YZ$$

Basically what we're doing is first trying to construct x (and marking when atleast one 1 is placed) but for each character in x, we are adding a variable (Y) so that we know we need to add atleast that many characters in y. ■

This page is for additional scratch work!

ECE 374 B Language Theory: Cheatsheet

1 Languages and strings

Languages

Definitions

- An *alphabet* Σ is a **finite** set of symbols.
- A *string* in Σ^* is a **finite** sequence of symbols in Σ.
- A *language* is L is a set of strings over some alphabet.

All languages represent mathematical problems.
Example: multiplication of two integers:

$$L_{MULT2} = \left\{ \begin{array}{lll} 1 \times 1|1, & 1 \times 2|2, & 1 \times 3|3, \ldots \\ 2 \times 1|2, & 2 \times 2|4, & 2 \times 3|6, \ldots \\ \vdots & \vdots & \vdots \\ n \times 1|n, & n \times 2|2n, & n \times 3|3n, \ldots \end{array} \right\} \quad (1)$$

Language operations

- For languages A, B the *concatenation* of A, B is $AB = \{xy \mid x \in A, y \in B\}$.
- For languages A, B, their *union* is $A \cup B$, *intersection* is $A \cap B$, and *difference* is $A \setminus B$ (also written as $A - B$).
- For language $A \subseteq \Sigma^*$ the *complement* of A is $\bar{A} = \Sigma^* \setminus A$.
- Σ^n is the set of all strings of length n.
- $\Sigma^* = \cup_{n \geq 0} \Sigma^n$ is the set of all strings over Σ.
- $\Sigma^+ = \cup_{n \geq 1} \Sigma^n$ is the set of non-empty strings over Σ.

Strings

Definitions

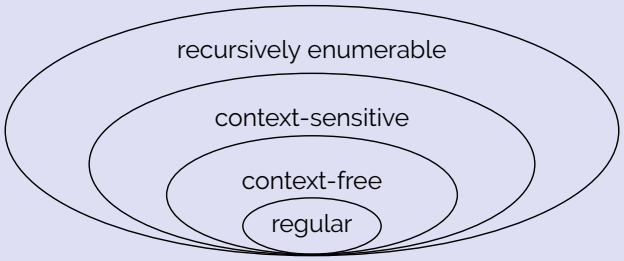
- The *length* of a string w (denoted by $|w|$) is the number of symbols in w.
- For integer $n \geq 0$, Σ^n is set of all strings over Σ of length n. Σ^* is the set of all strings over Σ.
- Σ^* is the set of all strings of all lengths including empty string.
- ε is a *string* containing no symbols.
- $\varnothing$ is the *empty set*. It contains no strings.

String operations

- If x and y are strings then xy denotes their concatenation. Recursively:
  - $xy = y$ if $x = \varepsilon$
  - $xy = a(wy)$ if $x = aw$
- v is *substring* of $w \iff$ there exist strings x, y such that $w = xvy$.
  - If $x = \varepsilon$ then v is a *prefix* of w
  - If $y = \varepsilon$ then v is a *suffix* of w
- A *subsequence* of a string $w = w_1 w_2 \ldots w_n$ is either a subsequence of $w_2 \ldots w_n$ or w_1 followed by a subsequence of $w_2 \ldots w_n$.
- If w is a string then w^n is defined inductively as follows: $w^n = \varepsilon$ if $n = 0$ or $w^n = ww^{n-1}$ if $n > 0$

2 Overview of language complexity

Overview



Grammar	Languages	Production Rules	Automaton	Examples
Type-0	recursively enumerable	$\gamma \to \alpha$ (no constraints)	Turing machine	$L = \{w \mid w \text{ is a } TM \text{ which halts}\}$
Type-1	context-sensitive	$\alpha A \beta \to \alpha \gamma \beta$	linear bounded nondeterministic Turing machine	$L = \{a^n b^n c^n \mid n > 0\}$
Type-2	context-free	$A \to \alpha$	nondeterministic pushdown automata	$L = \{a^n b^n \mid n > 0\}$
Type-3	regular	$A \to aB$	finite state machine	$L = \{a^n \mid n > 0\}$

Meaning of symbols:
- a - terminal
- A, B - variables
- α, β, γ - strings in $\{a \cup A\}^*$ where α, β are maybe empty, γ is never empty

[a]

[a] Table borrowed from Wikipedia: `https://en.wikipedia.org/wiki/Chomsky_hierarchy`

3 Regular languages

Regular language - overview

A language is regular if and only if it can be obtained from finite languages by applying

- union,
- concatenation or
- Kleene star

finitely many times. All regular languages are representable by regular grammars, DFAs, NFAs and regular expressions.

Regular expressions

Useful shorthand to denotes a language.
A *regular expression* $\mathbf{r}$ over an alphabet Σ is one of the following:
Base cases:

- $\varnothing$ the language $\varnothing$
- ε denotes the language $\{\varepsilon\}$
- a denote the language $\{a\}$

Inductive cases: If $\mathbf{r_1}$ and $\mathbf{r_2}$ are regular expressions denoting languages L_1 and L_2 respectively (i.e.,$L(\mathbf{r_1}) = L_1$ and $L(\mathbf{r_2}) = L_2$) then,

- $\mathbf{r_1} + \mathbf{r_2}$ denotes the language $L_1 \cup L_2$
- $\mathbf{r_1} \cdot \mathbf{r_2}$ denotes the language $L_1 L_2$
- $\mathbf{r_1^*}$ denotes the language L_1^*

Examples:

- 0^* - the set of all strings of 0s, including the empty string
- $(00000)^*$ - set of all strings of 0s with length a multiple of 5
- $(0+1)^*$ - set of all binary strings

Nondeterministic finite automata

NFAs are similar to DFAs, but may have more than one transition destination for a given state/character pair.

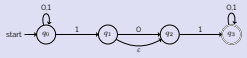
An NFA N *accepts a string* w iff some accepting state is reached by N from the start state on input w.

The *language accepted (or recognized)* by an NFA N is denoted $L(N)$ and defined as $L(N) = \{w \mid N \text{ accepts } w\}$.

A *nondeterministic finite automaton (NFA)* $N = (Q, \Sigma, s, A, \delta)$ is a five tuple where

- Q is a finite set whose elements are called *states*
- Σ is a finite set called the *input alphabet*
- $\delta : Q \times \Sigma \cup \{\varepsilon\} \to \mathcal{P}(Q)$ is the *transition function* (here $\mathcal{P}(Q)$ is the power set of Q)
- s and Σ are the same as in DFAs

Example:



- $Q = \{q_0, q_1, q_2, q_3\}$
- $\Sigma = \{0, 1\}$
- δ :

	ε	0	1
q_0	$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_1, q_2\}$	$\{q_2\}$	$\varnothing$
q_2	$\{q_2\}$	$\varnothing$	$\{q_3\}$
q_3	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$

- $s = q_0$
- $A = \{q_3\}$

For NFA $N = (Q, \Sigma, \delta, s, A)$ and $q \in Q$, the ε-reach(q) is the set of all states that q can reach using only ε-transitions.
Inductive definition of $\delta^* : Q \times \Sigma^* \to \mathcal{P}(Q)$:

- if $w = \varepsilon$, $\delta^*(q, w) = \varepsilon$-reach$(q)$
- if $w = a$ for $a \in \Sigma$, $\delta^*(q, a) = \varepsilon\text{reach}\left(\bigcup_{p \in \varepsilon\text{-reach}(q)} \delta(p, a)\right)$
- if $w = ax$ for $a \in \Sigma, x \in \Sigma^*$: $\delta^*(q, w) = \varepsilon\text{reach}\left(\bigcup_{p \in \varepsilon\text{-reach}(q)} \left(\bigcup_{r \in \delta^*(p,a)} \delta^*(r, x)\right)\right)$

Regular closure

Regular languages are closed under union, intersection, complement, difference, reversal, Kleene star, concatenation, etc.

Deterministic finite automata

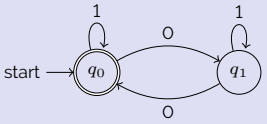
DFAs are finite state machines that can be represented as a directed graph or in terms of a tuple.

The *language accepted* (or recognized) by a DFA M is denoted by $L(M)$ and defined as $L(M) = \{w \mid M \text{ accepts } w\}$.

A *deterministic finite automaton (DFA)* $M = (Q, \Sigma, s, A, \delta)$ is a five tuple where

- Q is a finite set whose elements are called *states*
- Σ is a finite set called the *input alphabet*
- $\delta : Q \times \Sigma \to Q$ is the *transition function*
- $s \in Q$ is the *start state*
- $A \subseteq Q$ is the set of *accepting/final* states

Example:



- $Q = \{q_0, q_1\}$
- $\Sigma = \{0, 1\}$
- δ :

	0	1
q_0	q_1	q_0
q_1	q_0	q_1

- $s = q_0$
- $A = \{q_0\}$

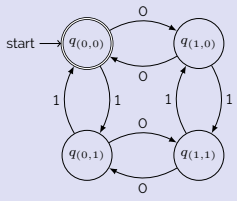
Every string has a unique walk along a DFA. We define the extended transition function as $\delta^* : Q \times \Sigma^* \to Q$ defined inductively as follows:

- $\delta^*(q, w) = q$ if $w = \varepsilon$
- $\delta^*(q, w) = \delta^*(\delta(q, a), x)$ if $w = ax$.

Can create a larger DFA from multiple smaller DFAs. Suppose

- $L(M_0) = \{w \text{ has an even number of 0s}\}$ (pictured above) and
- $L(M_1) = \{w \text{ has an even number of 1s}\}$.

$L(M_C) = \{w \text{ has even number of 0s and 1s}\}$

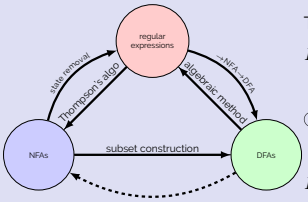


Suppose $M_0 = (Q_0, \Sigma, s_0, A_0, \delta_0)$ and $M_1 = (Q_1, \Sigma, s_1, A_1, \delta_1)$. Then

- $Q = Q_0 \times Q_1 = \{(q_0, q_1) \mid q_0 \in Q_0, q_1 \in Q_1\}$
- $s = (s_0, s_1)$
- $\delta : Q \times \Sigma \to Q$, where $\delta((q_0, q_1), a) = (\delta_0(q_0, a), \delta_1(q_1, a))$
- $A = \{(q_0, q_1) \mid q_0 \in A_0 \text{ and } q_1 \in A_1\}$

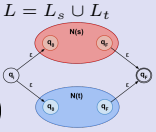
Regular language equivalences

A regular language can be represented by a regular expression, regular grammar, DFA and NFA.

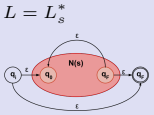


Thompson's algorithm:

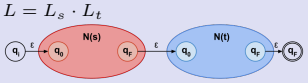
$L = L_s \cup L_t$



$L = L_s^*$



$L = L_s \cdot L_t$



Arden's rule: If $R = Q + RP$ then $R = QP^*$.

Fooling sets

Some languages are not regular (Ex. $L = \{0^n 1^n \mid n \geq 0\}$).

Two states $p, q \in Q$ are *distinguishable* if there exists a string $w \in \Sigma^*$, such that

$\delta^*(p, w) \in A$ and $\delta^*(q, w) \notin A$.

or

$\delta^*(p, w) \notin A$ and $\delta^*(q, w) \in A$.

Two states $p, q \in Q$ are *equivalent* if for all strings $w \in \Sigma^*$, we have that

$\delta^*(p, w) \in A \iff \delta^*(q, w) \in A$.

For a language L over Σ a set of strings F (could be infinite) is a *fooling set* or *distinguishing set* for L if every two distinct strings $x, y \in F$ are distinguishable.

4 Context-free languages

Context-free languages

A language is context-free if it can be generated by a context-free grammar. A context-free grammar is a quadruple $G = (V, T, P, S)$

- V is a finite set of *nonterminal (variable) symbols*
- T is a finite set of *terminal symbols* (alphabet)
- P is a finite set of *productions*, each of the form $A \to \alpha$ where $A \in V$ and α is a string in $(V \cup T)^*$ Formally, $P \subseteq V \times (V \cup T)^*$.
- $S \in V$ is the *start symbol*

Example: $L = \{ww^R | w \in \{0,1\}^*\}$ is described by $G = (V, T, P, S)$ where V, T, P and S are defined as follows:

- $V = \{S\}$
- $T = \{0, 1\}$
- $P = \{S \to \varepsilon \mid 0S0 \mid 1S1\}$
  (abbreviation for $S \to \varepsilon, S \to 0S0, S \to 1S1$)
- $S = S$

Pushdown automata

A pushdown automaton is an NFA with a stack.

The language $L = \{0^n 1^n \mid n \geq 0\}$ is recognized by the pushdown automaton:

A *nondeterministic pushdown automaton (PDA)* $P = (Q, \Sigma, \Gamma, \delta, s, A)$ is a **six** tuple where

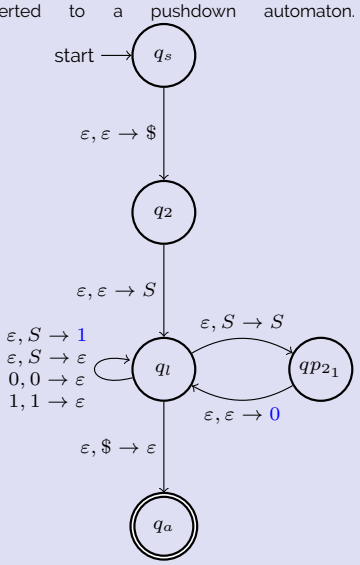
- Q is a finite set whose elements are called *states*
- Σ is a finite set called the *input alphabet*
- Γ is a finite set called the *stack alphabet*
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \to \mathcal{P}(Q \times (\Gamma \cup \{\varepsilon\}))$ is the *transition function*
- s is the start state
- A is the set of accepting states

In the graphical representation of a PDA, transitions are typically written as $\langle$input read$\rangle, \langle$stack pop$\rangle \to \langle$stack push$\rangle$.

A CFG can be converted to a pushdown automaton.

The PDA to the right recognizes the language described by the following grammar:

$S \to 0S|1|\varepsilon$



Context-free closure

Context-free languages are closed under union, concatenation, and Kleene star.

They are **not** closed under intersection or complement.